

Master Project Context & Coding Agent Brief

1. Project Summary

Build a modern bilingual web platform for learning and exploring Sanatan Dharma.

The product should prioritize:

- Accuracy
- Source transparency
- Beginner-friendly explanations
- English and Hindi
- Respect for multiple traditions
- Structured learning
- Bhagavad Gita reading
- Search
- Source-grounded AI assistance
- User bookmarks and learning progress

The platform should eventually become a trusted digital learning environment rather than a generic religious blog.

2. Core Product Promise

The platform should help users understand Sanatan Dharma through:

clear explanations + authentic sources + structured learning + modern technology

The system must clearly distinguish between:

- Original scripture
- Translation
- Commentary
- Editorial explanation
- AI-generated explanation

Never blur these categories.

3. Primary Audience

Primary audience:

Indian teenagers and adults who want to understand Sanatan Dharma, especially beginners.

Secondary users:

- Practicing Hindus
- Students
- People reconnecting with Sanatan traditions
- Advanced learners

Future:

- Children
 - Scholars
 - International audiences
 - Mobile app users
-

4. Languages

Initial languages:

- English
- Hindi

Also support:

- Sanskrit text
- IAST transliteration

The architecture must allow more languages later.

5. Core V1 Features

Version 1 should include:

- Homepage
- Learn section
- Beginner learning path
- Concepts
- Scripture library
- Bhagavad Gita
- Chapter pages
- Verse pages
- Deities

- Mantras
- Festivals
- Search
- Ask Dharma AI
- Authentication
- Bookmarks
- My Library
- Basic learning progress
- Source display
- Content reporting

Do not build unrelated large features during V1.

6. Out of Scope for Initial Build

Do not build initially:

- Community/forum
- Native mobile application
- Astrology
- Horoscope
- Temple booking
- Priest booking
- Marketplace
- Full Panchang system
- Live streaming
- Complex gamification
- Every scripture
- Every commentary
- Large audio library

Do not expand scope unless explicitly requested.

7. Recommended Technical Stack

Use:

Frontend

- Next.js
- React
- TypeScript
- App Router

Styling

- Tailwind CSS
- shadcn/ui where useful

Backend

- Next.js server layer
- Route Handlers
- Server Actions where appropriate

Database

- PostgreSQL
- Supabase

ORM

- Drizzle ORM

Validation

- Zod

Authentication

- Supabase Auth

Initial login:

- Email
- Google

Search

Start with:

- PostgreSQL full-text search

Support later:

- pgvector semantic search

AI/RAG

- Provider abstraction

- Retrieval-Augmented Generation
- PostgreSQL + pgvector
- Hybrid keyword/vector retrieval
- Source citation validation

Hosting

- Vercel

Source Control

- Git
- GitHub

Testing

- Vitest
- Playwright

Monitoring

- Sentry or equivalent
-

8. Architecture Principle

Keep the system:

simple enough to build now, structured enough to grow later

Do not prematurely add:

- Kubernetes
 - Microservices
 - Separate backend service
 - Multiple databases
 - Elasticsearch cluster
 - Custom auth
 - Complex infrastructure
-

9. Project Structure

Preferred high-level structure:

```
```text id="2i7y0e" src/ |—— app/ |—— components/ |—— features/ |—— lib/ |—— server/ |—— db/
|—— schemas/ |—— services/ |—— types/ |—— styles/
```

Exact structure may evolve, but business logic should not be embedded randomly inside UI components.

---

#### # 10. Service Layer Rule

Prefer:

```
```text id="y8l3sv"
UI / Route Handler
↓
Validation
↓
Service
↓
Database / Provider
```

Avoid directly calling database logic from many unrelated components.

11. Database Principle

The system should store meaning and provenance, not just blocks of text.

For important religious content, the system should be able to identify:

- What is this content?
- What scripture or source does it come from?
- Who translated it?
- Who commented on it?
- Which tradition does the interpretation belong to?
- Has it been verified?
- Can AI retrieve it?
- Can the user view the source?

12. Critical Data Separation

Never combine these in the same field:

These must remain separate entities.

13. Main Database Entities

The architecture should support:

- profiles
- roles
- user_roles
- scriptures
- scripture_localizations
- scripture_sections
- scripture_section_localizations
- verses
- transliterations
- translations
- translators
- commentators
- commentaries
- editorial_explanations
- sources
- source_reviews
- licenses
- traditions
- tradition_localizations
- concepts
- concept_localizations
- deities
- deity_localizations
- mantras
- mantra_localizations
- festivals
- festival_dates
- festival_localizations
- tags
- content_tags
- content_relationships
- learning_paths
- learning_path_localizations
- learning_modules
- user_learning_progress
- bookmarks
- reading_history

- ai_conversations
- ai_messages
- ai_message_citations
- knowledge_chunks
- content_reviews
- content_reports
- audit_logs

Do not create all tables blindly at once. Introduce them as features require them while respecting this model.

14. Religious Accuracy Rules

The project must prioritize accuracy over content quantity.

Important rules:

- Do not fabricate scripture.
- Do not fabricate Sanskrit.
- Do not fabricate verse numbers.
- Do not fabricate commentaries.
- Do not present one tradition as universally correct.
- Attribute interpretations.
- Distinguish traditional and academic perspectives when necessary.
- Display sources for important claims.
- Do not publish draft or unverified content as verified.

15. Source Hierarchy

Preferred source hierarchy:

1. Primary scriptures
2. Classical commentaries
3. Established traditional institutions/publishers
4. Academic sources
5. General websites only as discovery aids

General blogs/social media should not be treated as authoritative evidence.

16. Copyright Rule

Never assume online text or imagery is reusable.

Track:

- Source
- Copyright status
- License
- Attribution requirements

Prefer:

- Public domain
- Open license
- Original platform material
- Properly licensed content

17. Visual Direction

Design theme:

****Modern Indian Spiritual****

The UI should feel:

- Calm
- Trustworthy
- Modern
- Respectful
- Premium
- Easy to read

Use:

- Warm cream/off-white backgrounds
- Muted saffron/orange accents
- Deep maroon/brown accents
- Charcoal text
- Muted gold sparingly
- Generous whitespace
- Strong Devanagari typography
- High-quality imagery

Avoid:

- Excessive saffron
- Flashy gradients
- Religious clip-art

- Sacred symbols as generic UI controls
- Busy backgrounds
- Overdecorated temple themes

18. UX Principles

Every major page should be:

- Mobile responsive
- Accessible
- Easy for beginners
- Source transparent
- Bilingual
- Fast

The user should be able to understand what a page is about quickly.

19. Main Navigation

Desktop concept:

```text id="46pl8c"

Logo

Learn

Scriptures

Explore ▼

Ask Dharma

Search

Language

Library

Profile

Explore may include:

- Deities
- Mantras
- Festivals
- Traditions
- Philosophy

Mobile primary navigation:

---

## # 20. Homepage Priorities

Homepage should include:

- Hero
- Start Learning
- Ask Dharma
- Global search
- Start Your Journey cards
- Core concepts
- Featured Bhagavad Gita section
- Daily shloka
- Upcoming festival
- Explore section
- Trust/source explanation

The homepage must explain why users should trust the platform.

---

## # 21. Learning Structure

Beginner learning path:

```text id="d6oiqw"

What is Sanatan Dharma?

↓

Dharma

↓

Karma

↓

Atman

↓

Brahman

↓

Samsara

↓

Moksha

↓

Yoga

↓

Bhakti

↓

Introduction to Bhagavad Gita

Do not make learning feel overly gamified.

22. Concept Page Structure

Concept pages should support:

- Sanskrit term
- Simple explanation
- Deeper explanation
- Scriptural references
- Different interpretations where needed
- Common misunderstandings
- Related concepts
- Sources
- Ask Dharma

23. Bhagavad Gita Priority

Bhagavad Gita is a flagship experience.

Required hierarchy:

```text id="mcban9" Bhagavad Gita ↓ Chapter ↓ Verse

Each verse page should eventually support:

- Reference
- Sanskrit
- IAST transliteration
- Word-by-word meaning
- English translation
- Hindi translation
- Simple explanation
- Commentary
- Tradition labels
- Related concepts
- Sources
- Previous/next navigation

- Bookmark
- Ask Dharma

---

## # 24. Search Requirements

Search must support:

- English
- Hindi
- Sanskrit
- IAST
- Alternative spellings
- Aliases
- Exact verse references

Examples that should connect appropriately:

```text id="nyp9z0"

Krishna

Kṛṣṇa

कृष्ण

Search is a core product feature and should be built before production AI.

25. Ask Dharma Principle

The AI model is not the source.

The AI should help users understand approved sources.

Core rule:

Retrieve first. Answer second.

26. RAG Pipeline

Preferred flow:

```text id="ya0udh" User Question ↓ Validation ↓ Language Detection ↓ Intent Classification ↓ Query Expansion ↓ Keyword Search + Vector Search ↓ Metadata Filters ↓ Reranking ↓ Context Assembly ↓ LLM ↓ Citation Validation ↓ Structured Response

---

#### # 27. Ask Dharma Allowed Sources

Production retrieval may use:

- Verified scripture
- Approved translations
- Approved commentaries
- Verified concept pages
- Verified deity pages
- Verified festival pages
- Verified mantra pages
- Approved academic sources

Do NOT retrieve production answers from:

- Draft content
- Rejected sources
- Internal notes
- Restricted source material unless permitted
- Unverified AI-generated content

---

#### # 28. AI Answer Rules

AI must:

- Cite relevant sources
- Admit uncertainty
- Correct false premises
- Distinguish scripture from interpretation
- Represent traditions fairly
- Avoid fake confidence
- Avoid claiming religious authority
- Match English/Hindi user language
- Prefer retrieved Sanskrit instead of regenerating it

---

#### # 29. Invalid Scripture Handling

Example user prompt:

```
```text id="ch49se"
Explain Bhagavad Gita 25.90
```

The system must not hallucinate an answer.

It should detect that the reference cannot be found and respond accordingly.

This behavior is a launch-critical requirement.

30. Tradition Handling

If the user asks generally:

Represent relevant major views where they meaningfully differ.

If the user asks:

```
```text id="xwcl3e" Explain according to Advaita
```

```
then prioritize Advaita sources while clearly labeling the perspective.
```

```
Do not rank traditions as superior/inferior.
```

```

```

```
31. Historical Questions
```

```
Where traditional chronology and modern scholarship differ:
```

```
Present them separately.
```

```
Use labels such as:
```

```
Traditional perspective
```

```
Academic/historical perspective
```

```
Do not invent false certainty.
```

```

```

## # 32. User Accounts

Anonymous users should be able to read most public content.

Authentication is required for:

- Bookmarks
- Library
- Progress
- Saved AI conversations
- Preferences

Do not force login before users can understand the product.

---

## # 33. Security Rules

Must implement:

- Server-side authorization
- Input validation
- Secure cookies
- Row Level Security
- Rate limiting
- No secrets in browser
- Protected admin routes
- Secure upload validation
- User-data isolation

Frontend visibility is not security.

---

## # 34. Content Workflow

Content should flow approximately:

```text id="g7ox6g"

Draft

↓

Source Check

↓

Editorial Review

↓

Relevant Subject Review

↓

Verified

↓
Published

AI-generated text must not automatically become permanent verified content.

35. Admin Requirements

Admin area should eventually support:

- Scriptures
- Verses
- Translations
- Commentaries
- Concepts
- Deities
- Mantras
- Festivals
- Sources
- Reviews
- Error reports
- Users
- Audit logs

Do not build the entire admin platform before core public content works.

36. Development Order

Build in this order:

1. Project setup 2. Design system 3. Global layout 4. Homepage 5. Database foundation 6. Concepts 7. Scriptures 8. Bhagavad Gita 9. Deities 10. Mantras 11. Festivals 12. Relationships 13. Authentication 14. Bookmarks 15. Learning paths 16. Search 17. AI retrieval 18. Ask Dharma 19. Admin 20. Testing 21. Launch

37. First Vertical Slice

The first complete feature should be:

Karma Concept Page

It should test:

- Database record
- English localization
- Hindi localization
- Sanskrit term
- Source metadata
- Related content
- Responsive page
- SEO

Do not start by building dozens of content pages.

38. Second Vertical Slice

Build:

****Bhagavad Gita 2.47****

This should test:

- Scripture
- Chapter
- Verse
- Sanskrit
- Transliteration
- Translation
- Commentary
- Sources
- Relationships
- Verse reader UX

39. Third Vertical Slice

Build:

****Krishna deity page****

This should test:

- Deity data model
- Images
- Scripture links
- Mantra relationships

- Festival relationships

40. Fourth Vertical Slice

Build:

****Janmashtami festival page****

This should test:

- Festival content
- Date data
- Regional variation
- Deity relationship
- Sources

41. Fifth Vertical Slice

Build first AI use case:

****Ask Dharma → Explain Bhagavad Gita 2.47****

Only after:

- Gita 2.47 exists
- Sources exist
- Search works
- Retrieval works
- Citation metadata works

42. Coding Agent Rules

Before editing code, the coding agent should:

1. Inspect the current repository.
2. Read relevant project documentation.
3. Understand existing patterns.
4. Identify affected files.
5. Produce a concise implementation plan for substantial tasks.
6. Implement the smallest complete feature.
7. Run relevant checks.
8. Fix failures before claiming completion.

43. Scope Rule for Coding Agents

Do not modify unrelated files.

Do not perform large refactors unless necessary.

Do not introduce new libraries without a clear reason.

Do not change core architecture without explaining why.

44. Dependency Rule

Before installing a new package:

Check whether the project already has a suitable solution.

Prefer fewer dependencies.

Libraries should be:

- Maintained
- Appropriate
- Necessary

45. Hardcoding Rule

Avoid hardcoding religious content directly inside page components.

Religious content should come from structured data/database except for temporary development seed content.

Do not permanently embed scripture text directly in JSX.

46. TypeScript Rule

Use strong typing.

Avoid unnecessary:

```
```text id="ax6t9x"  
any
```

Use Zod or proper interfaces/types for external data.

---

## 47. Validation Rule

All user-controlled inputs must be validated server-side.

Examples:

- Search query
  - Bookmark request
  - AI request
  - Profile update
  - Admin content
  - File upload
- 

## 48. Error Handling

Every major feature should include:

- Loading state
- Empty state
- Error state

Do not leave blank screens on failure.

---

## 49. Accessibility Rule

Build accessibility into components from the beginning.

Use:

- Semantic HTML
- Keyboard navigation
- Visible focus
- Proper labels
- Alt text
- Accessible modals

- Adequate contrast
- 

## 50. Responsive Rule

Every feature must be tested on:

- Mobile
- Tablet
- Desktop

Do not build desktop-only layouts and “fix mobile later.”

---

## 51. Testing Definition of Done

A feature is not complete until:

- Implementation works
  - Type check passes
  - Build passes
  - Relevant tests pass
  - Mobile checked
  - Loading/error states work
  - Permissions checked where relevant
  - Sources appear correctly where relevant
- 

## 52. Critical Regression Tests

Permanent regression coverage should eventually include:

- Homepage
  - Karma
  - Bhagavad Gita chapters
  - Bhagavad Gita 2.47
  - English/Hindi switching
  - Search
  - Login
  - Bookmark
  - Ask Dharma exact verse
  - Fake verse rejection
-

## 53. Deployment Environments

Maintain:

```text id="m3hgnn" Development Staging Production

Staging should resemble production closely.

Do not test dangerous migrations directly in production.

54. Production Safeguards

Before launch ensure:

- HTTPS
- Backups
- RLS
- Secure auth
- Rate limits
- Monitoring
- Analytics
- Legal pages
- AI feature flag
- Search index
- AI index
- Rollback plan

55. Feature Flags

Use feature flags for risky or experimental functionality.

Especially:

```text id="vj9qjg"

Ask Dharma

Dark Mode

Advanced Commentary

Audio

Ask Dharma should be independently disableable.

## 56. Main Engineering Principle

Do not optimize for the largest possible future system.

Optimize for:

**a correct, maintainable, high-quality MVP that can evolve**

---

## 57. Main Product Principle

Do not optimize for maximum quantity of Sanatan Dharma content.

Optimize for:

**trust + understanding + provenance**

---

## 58. Agent Behavior When Requirements Are Unclear

Prefer existing project documentation.

Do not invent major product requirements.

For minor implementation details, choose the simplest solution consistent with:

- PRD
  - UI/UX specification
  - Technical requirements
  - Database schema
  - AI architecture
- 

## 59. Documentation Priority

If documents appear to conflict, use this order:

1. Latest explicit task instruction
2. Master Project Context
3. Product Requirements Document

4. Technical Requirements
5. Database Schema
6. AI Architecture
7. API Specification
8. UI/UX Specification
9. Development Roadmap

Flag meaningful architectural conflicts instead of silently choosing incompatible approaches.

---

## 60. Immediate Next Development Objective

Do NOT build the whole application in one request.

The immediate objective is:

### Phase 0 — Project Foundation

Set up:

- Next.js
- TypeScript
- Tailwind CSS
- Git
- Project structure
- Supabase configuration placeholders
- Drizzle
- Zod
- Environment template
- Basic home route
- Build/lint validation

No production AI yet.

No large content import yet.

No full database schema yet.

---

## 61. First Coding Agent Instruction

When beginning development, use an instruction similar to:

```
```text id="11xi4v" You are working on the Sanatan Dharma learning platform.
```

Read the project's Master Project Context and all relevant architecture documents before making changes.

Your current task is Phase 0 only.

Set up the project foundation using: - Next.js App Router - TypeScript - Tailwind CSS - Drizzle ORM - Zod - Supabase-ready environment configuration

Requirements: - Keep architecture simple. - Do not build AI yet. - Do not build authentication yet. - Do not import large religious datasets. - Create a clean maintainable folder structure. - Add .env.example. - Ensure secrets are never committed. - Add a minimal homepage proving the app works. - Run lint/type/build checks. - Fix errors before completion.

At the end, report: 1. Files created/changed 2. Dependencies added 3. Commands run 4. Check results 5. Any decisions or blockers ``

62. Status

Master Project Context & Coding Agent Brief:

Complete

The planning phase is complete enough to begin implementation.